

Agentic QA Automation Pipeline

User stories in. Running browser tests out.

Product overview — features, engineering work, and the problems it solves.

Prepared for the engineering and QA team · October 2026

Contents

1. Summary	3
At a glance	3
2. The problems it solves	3
Writing end-to-end tests is slow and needs specialist skills	3
Tests rot, and failures are opaque	3
Nobody knows what is actually covered	3
QA progress is invisible to non-engineers	4
Flaky tests erode trust	4
Full suites are too slow to run on every change	4
3. How it works	5
A note on how the tests are written	5
4. Feature catalogue	5
Authoring and generation	5
Running	6
Diagnosis and reporting	6
Beyond functional testing	6
5. Architecture and technology	7
Testing more than one application	7
6. Current state	7
Engineering work completed	8
7. Limitations and next steps	9
Known limitations	9

Proposed next steps	9
8. In one paragraph.....	9

1. Summary

The Agentic QA Automation Pipeline turns a written user story into executable, maintainable browser tests — and then runs, reports on, schedules and repairs them from a single local web app.

A tester types an application URL and a few acceptance criteria. The pipeline drives a real browser to study the application, writes Gherkin scenarios, generates the page objects and step definitions behind them, runs the suite across seven browser and device profiles, and produces reports for both engineers and managers. When a test later breaks, it explains the failure in plain English and proposes the fix.

It runs entirely on local machines against a Claude Code CLI subscription. There are no API keys to manage and no per-test cost, which is what makes an exploratory, regenerate-often workflow affordable.

At a glance

What it is	A local web app plus a Playwright + Cucumber test framework
Who uses it	QA engineers writing tests; managers reading results
Input	An application URL and acceptance criteria — or a recorded browser session
Output	Gherkin features, page objects, step definitions, and four report formats
Runs on	Chromium, Firefox, WebKit, plus four emulated tablet and phone profiles
Cost model	Claude Code CLI subscription — no API keys, no per-test charge

2. The problems it solves

Each of the following is a problem the team actually had. The feature that answers it is named alongside.

Writing end-to-end tests is slow and needs specialist skills

A single login scenario means learning the DOM, choosing resilient selectors, writing a page object and wiring step definitions. That is hours of specialist work per feature, which is why coverage tends to stop at the happy path.

Answer: the generation pipeline writes all four artefacts from the acceptance criteria, having first driven the live application to find the real selectors.

Tests rot, and failures are opaque

A renamed button breaks a test, and the report says only that a locator timed out. Someone has to open a trace and work out whether the application broke or the test did.

Answer: Explain Failure translates the error into plain English; Heal with Claude reads the failure, the screenshot and the live page, then proposes a corrected locator for review.

Nobody knows what is actually covered

A green suite says the tests that exist passed. It says nothing about the pages and buttons no one ever wrote a test for.

Answer: the exploratory agent crawls the running application read-only, compares what it finds against the suite, and ranks the gaps — weighting irreversible actions highest, because untested and irreversible is the worst combination.

QA progress is invisible to non-engineers

Playwright and Allure reports are built for engineers. A manager asking "what works today?" gets a wall of traces.

Answer: alongside those, every run produces a per-feature Markdown summary and a formatted Excel workbook with one tab per feature and a totals sheet.

Flaky tests erode trust

A test that passes on Monday and fails on Tuesday is worse than no test, because it trains the team to ignore red.

Answer: per-test history across runs surfaces tests whose results alternate, so flakiness is visible as a pattern rather than argued about case by case.

Full suites are too slow to run on every change

Running everything on every commit does not scale; running nothing is worse.

Answer: the impact map reads the git diff and maps changed files to the features they affect, so only the relevant subset runs.

3. How it works

The pipeline has four stages. Each is usable on its own.

Stage	What happens	Produced
1 — Describe	Enter the URL and acceptance criteria, or record yourself using the app. Spec Doctor reviews the criteria for testability first.	A user story file
2 — Generate	The agent opens the application in a real browser, reads the live DOM, and writes the tests. Existing step phrases are reused rather than duplicated.	Gherkin feature, step definitions, page objects
3 — Run	Run one feature, one tag, only what a code change affects, only what failed last time, or the whole suite across all seven profiles.	Live step-by-step view, screenshots, video, traces
4 — Report & repair	Four report formats are written. Failures can be explained or repaired by the agent, with the change shown for approval.	Reports, proposed fixes

A note on how the tests are written

Generated tests follow the same conventions a careful engineer would use by hand, because the generator is held to them explicitly:

- Gherkin is the only authoring path — business-readable scenarios, never raw test scripts.
- Every selector lives on a page object. A step definition that contains a raw locator is treated as a defect, so a UI change touches one file rather than fifty.
- Elements are located by role and accessible name first, which is both more resilient than CSS and a passive accessibility check.
- Step definitions are scoped by tag, so two applications can use the same wording without colliding.
- An acceptance criterion that cannot be verified fails loudly rather than being quietly dropped — a green suite that proves nothing is treated as worse than an honest red one.

4. Feature catalogue

Authoring and generation

Feature	What it does
Story to tests	Generates Gherkin, step definitions and page objects from acceptance criteria.
Spec Doctor	Scores acceptance criteria against a testability rubric — vague verbs, missing negative cases, unstated preconditions — before any generation runs.
Test Recorder	Click through the application; the session becomes a Gherkin scenario, then real implementations and page objects.
Scaffold missing steps	Finds step phrases with no implementation and writes them, creating the page object they wrap.
Step reuse index	Knows every existing step phrase and its tag scope, so generation reuses wording instead of duplicating it.

Feature	What it does
Coverage gap closer	Lists acceptance criteria with no matching scenario and drafts the missing one.
Tag editor	Edit scenario tags from the UI for selective runs.

Running

Feature	What it does
Seven run profiles	Chromium, Firefox and WebKit, plus emulated iPad Pro 11, Galaxy Tab S4, iPhone 14 and Galaxy S24.
All browsers at once	The whole suite across every profile in a single run.
Live step timeline	Each Given/When/Then lights up as it executes, with screenshots.
Selective runs	By feature, by tag, by git impact, or only the tests that failed last time.
Scheduled runs	Every N minutes, daily or weekly — streamed live, with overlap protection and optional webhook alerts.
Visual regression	Snapshot a page and compare it against a committed baseline.

Diagnosis and reporting

Feature	What it does
Heal with Claude	Reads the failure, screenshot and live page, then proposes a fix for review.
Explain failure	Plain-English cause, readable by someone who has never opened a trace.
Flakiness detection	Per-test history across runs identifies tests that alternate.
Four report formats	Playwright HTML and Allure for engineers; per-feature Markdown and a per-site Excel workbook for everyone else.
Screenshot gallery	Every run's screenshots in one place, with passwords blurred automatically at capture time.

Beyond functional testing

Feature	What it does
Exploratory agent	Crawls the running application read-only and ranks untested surface, weighting irreversible actions highest.
Security scan	Headers, cookie flags and common misconfigurations, graded A–F.
Performance	Lighthouse run with configurable budgets.
API testing	JSON-defined request suites run alongside the browser tests.

5. Architecture and technology

Everything runs locally. No test data or application credentials leave the machine.

Layer	Technology	Role
Browser automation	Playwright 1.60	Drives Chromium, Firefox and WebKit; device emulation; traces and video
Test authoring	Cucumber / Gherkin via playwright-bdd	Business-readable scenarios compiled to runnable tests
Design pattern	Page Object Model	Selectors isolated from test logic
The agent	Claude Code CLI	Generation, healing, explanation, scaffolding — no API key
Agent browser access	Playwright MCP server	Lets the agent inspect the live application while writing tests
Application	Node.js and Express	Local web UI, 52 endpoints, live streaming over SSE
Reporting	Allure, Playwright HTML, ExcelJS	Engineer-facing and manager-facing outputs

Testing more than one application

Each application under test owns a self-contained folder holding its own features, page objects and user stories. Adding a second or third product never writes into the first one's files, and a run can be scoped to one application. Entering a new URL creates its folder automatically and shows where the code will land before anything is generated.

Tag-scoped step definitions are what make this safe: two applications can both have a step worded "I log in" without colliding.

6. Current state

Measured from the repository as it stands.

Metric	Value
Applications under test	3
Feature files / scenarios	7 features, 41 scenarios
Page objects	12
Tests collected across all profiles	287
Run profiles configured	7
Application endpoints	52
Supporting engine modules	17

The suite runs green on Chromium, and type checking and compilation are clean. Destructive scenarios — ones that would send a real order or create a real account — are tagged and excluded from every run started in the UI.

Engineering work completed

- Migrated from a single-application layout to self-contained per-application folders, with the original suite moved across and left passing.
- Added recording as a way to start a brand-new application, not only to extend an existing one.
- Made generation produce page objects rather than inline selectors, holding generated code to the same rule as hand-written code.
- Split the Excel export into one workbook per application, so running one product no longer blanks another's recorded results.
- Automatic password blurring on every screenshot, applied at capture time.
- Hardened the runner: overlap protection on scheduled runs, validation of run arguments, and a warning when the server is running older code than what is on disk.

7. Limitations and next steps

Stated plainly, because knowing where the edges are is part of trusting the tool.

Known limitations

Limitation	Detail
Generated selectors are a first draft	The agent inspects the live application, but a generated test may still need one locator corrected. Healing is the intended loop, not an exception.
Generation is not instant	A full scaffold takes five to ten minutes, because the agent drives the real application before answering. Progress is shown while it works.
Device profiles are emulation	Viewport, user agent and touch on a desktop engine — not a real iOS or Android device.
Tests depend on a live application	A slow or changing third-party environment produces failures that are environmental rather than genuine.
One runner	Playwright only. Supporting another framework would mean a second implementation of the runner, reporting and generation paths.

Proposed next steps

Priority	Item	Why
1	Record and replay network traffic	Removes dependence on a live environment; turns a slow suite into a fast, deterministic one for CI.
2	Accessibility checks	The one testing dimension with no coverage today; fits the existing scan-and-report pattern.
3	Move credentials out of scenario files	Test credentials are currently in plain text — a habit worth correcting before it reaches a real environment.
4	Commit visual baselines	Baselines are written to a generated folder, so visual checks cannot currently mean anything in CI.
5	Shared login state	Tests that are not testing login could skip it, shortening every run.

8. In one paragraph

This pipeline removes the two bottlenecks in end-to-end testing: the cost of writing tests, and the cost of keeping them alive. Acceptance criteria become working tests in minutes instead of hours; a broken test explains itself and proposes its own repair; and coverage gaps are discovered by an agent that explores the application rather than waiting for someone to notice. It runs on the machines the team already has, against a subscription they already pay for, and it reports in a form both engineers and managers can read.